SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. *(BL3)*

Assessment Tool Weightage: 40%

Duration :60 Mins
On Class

QUESTIONS: 5 **Total Marks:50**

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

I V G Abbiramy - 192524097 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

V G Abbiramy

CO-1 → Analytical

i) $w = x * y - z + 20$

ii) Lexical Analysis:

w	→ Identifier	z	→ Identifier
=	→ Operator	+	→ Operator
x	→ Identifier	20	→ Identifier
*	→ Operator		
y	→ Identifier		
-	→ Operator		

ii) Syntax Analysis:

$E \rightarrow E + T$

$E \rightarrow E - T$

$T \rightarrow T * F$

$F \rightarrow id / num$

iii) Semantic Analysis:

$x * y \rightarrow$ Real x Real = Real

$(x * y) - z \rightarrow$ Real - Real = Real

$((x * y) - z) + 20 \rightarrow$ Real + integer = Real

iv) Intermediate Code:

$t1 = x * y$

$t2 = t1 - z$

$t3 = t2 + 20$

$w = t3$

v) Code Optimization:

$t1 = x * y$

$t1 = t1 - z$

$w = t1 + 20$

vi) Target code:

```
LOAD x
MUL y
SUB z
ADD #20
STORE w
```

2) i) $1(0+1)^*1+1$

ii) $(0+1)^*01(0+1)^*$

iii) 1^*01^*

iv) $(0+1)^*1(0+1)^*1(0+1)^*$

v) $\epsilon+1+(0+1)^*1$

3) i) $L=(\epsilon+aa^*)^* = \{\epsilon, a, aa, aaa, aaaa, \ldots\}$

ii) $L \Rightarrow (a^*+b^*)^*abb = \{abb, aabb, bbabb, ababb\}$

iii) $L \Rightarrow (a^*b^*)^*aba = \{aba, aaba, bbaba, aaababa\}$

iv) $L \Rightarrow (a+ab)^* = \{\epsilon, a, ab, aa, aab, abab\}$

v) $L \Rightarrow (aba)^*+(a^*b^*)^* = \{\epsilon, aba, abaaba, aab, aaabbb, abab\}$

4) i) Lex Regular Expression:

$$R.E = [a-zA-Z][a-zA-Z0-9]^*$$

Code:

```
%{
#include <stdio.h>
%}
%%
[a-zA-Z][a-zA-Z0-9]*
{
  printf("%s is a valid identifier\n", yytext); }
[0-9]+
{
printf("%s is a number\n", yytext); }
```

Date:

```
[ \t\n]+
{
   printf("%s is an invalid token\n", yytext); }
%%
int yywrap()
{
   return 1;
}
int main()
{
   printf("Enter the input :\n");
   yylex();
   return 0;
}
```

5) i) $(0^*1^*)^* = (0+1)^*$

→ 0^*1^* generates any no. of 0's followed by 1's.

→ Repeating it (*) allows every combination of 0 & 1.

∴

$(0^*1^*)^* = (0+1)^*$

ii) $(0+1)^*(0+1)^* = (0+1)^*$

→ Kleene star is closed under concatenation.

→ ∴

$(L^*L^*) = L^*$

iii) $(0^*)^* = 0^*$

→ Applying star twice does not change the lang.

iv) $(0+1)^*0(0+1)^* + (0+1)^*1(0+1)^* = (0+1)^+$

→ first expression contains strings having atleast one 0.

v) $\Sigma + 0(0)^* = 0^*$

→ $0^* = \Sigma + 0 + 00 + 000 + \ldots$

$P(0)^* = 0 + 00 + 000 + \dots$

$\therefore$

$\Sigma + 0(0)^* = \Sigma + 0 + 00 + 000 \dots = 0^*$

Hence proved.